
Multi-Agent Systems

Technical Notes and Notebook Guide

Week 7 · Lectures A & B · Notebook: `Week7_MultiAgent.ipynb`

Topics covered: PettingZoo API · Simple Spread (MPE) · Independent Learners · Cooperative failure & non-stationarity · CTDE / MADDPG sketch · LLM-driven agents · Bridge to GABM

Physics thread. A single spin in an external field $H = -h\sigma_i$ is the single-agent MDP. Moving to N interacting spins $H = -J \sum_{ij} \sigma_i \sigma_j - h \sum_i \sigma_i$ is the multi-agent extension. Mean-field theory is the first diagnostic tool; it reveals when agent-agent correlations matter. CTDE corresponds to solving the mean-field equations at training time, then deploying each spin with its effective field h_i^{eff} only.



1 Theoretical Background	2
1.1 From Single-Agent MDP to Multi-Agent Game	2
1.2 Nash Equilibrium	2
1.3 Cooperative MARL and Free Energy	2
1.4 Independent Learners	3
1.5 Centralised Training, Decentralised Execution (CTDE)	3
2 PettingZoo API — Notebook Sections 0 & 1	3
2.1 The <code>simple_spread</code> environment	3
3 Independent Learners — Notebook Sections 2, 3 & 4	5
3.1 Theory recap (Section 2)	5
3.2 Parameter-sharing IL (Section 3)	5
3.3 Separate-policy IL — manual REINFORCE loop (Section 4)	6
4 Diagnosing Cooperative Failure — Notebook Section 5	6
4.1 Three diagnostic metrics	6
5 CTDE Conceptual Sketch — Notebook Section 6	7
5.1 Architecture comparison	7
6 Bridge to GABM — Notebook Section 7	8
6.1 From reward to persona	8
7 Summary and What's Next	9
7.1 Key results from the notebook	9
7.2 Exercises	9
7.3 What comes next	9
7.4 References	10
A Notation Reference	10
B SuperSuit Wrapper Chain	10

■

1.1 From Single-Agent MDP to Multi-Agent Game

A single-agent Markov Decision Process is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. The agent's goal is to find the policy π^* maximising $\mathbb{E}_\pi [\sum_t \gamma^t r_t]$, solved via the Bellman optimality equation:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a').$$

This formulation rests on three assumptions that break in the multi-agent setting:

1. **Stationarity**: P and R do not change during learning.
2. **Full observability**: the state s is Markovian.
3. **Single objective**: one agent, one reward signal.

Non-stationarity — the core problem. In a multi-agent system with N agents, the transition kernel perceived by agent i is the *marginalisation*:

$$P_i(s' | s, a_i) = \sum_{a_{-i}} P(s' | s, \mathbf{a}) \pi_{-i}(a_{-i} | s).$$

This effective kernel changes every time any π_j ($j \neq i$) is updated. Therefore, from agent i 's viewpoint, the environment is non-stationary even if the underlying MDP has fixed dynamics.

1.2 Nash Equilibrium

The solution concept that replaces π^* in the multi-agent setting is the Nash Equilibrium.

Nash Equilibrium. A strategy profile $(\pi_1^*, \dots, \pi_N^*)$ is a Nash Equilibrium if, for every agent i and every alternative policy π'_i ,

$$V_i(\pi_i^*, \pi_{-i}^*) \geq V_i(\pi'_i, \pi_{-i}^*),$$

where $V_i(\pi_i, \pi_{-i}) = \mathbb{E}_{\pi_i, \pi_{-i}} [\sum_t \gamma^t r_i^t]$. No agent can increase its return by *unilaterally* deviating.

Key insight: NE $\neq$ social optimum. The Prisoner's Dilemma shows that rational individual behaviour (NE) can lead to a collectively suboptimal outcome. In cooperative MARL this means agents may converge to a NE that is far from the Pareto frontier. Reward shaping, CTDE, and explicit communication are the main tools to close this gap.

1.3 Cooperative MARL and Free Energy

In a cooperative task all agents share the same reward $r = r_i$ for all i . The collective objective is:

$$\max_{\pi_1, \dots, \pi_N} \mathbb{E} \left[\sum_t \gamma^t r^t \right].$$

When agents use entropy-regularised RL (as in PPO), each agent effectively minimises a local free energy:

$$\mathcal{F}_i = \mathbb{E}_{o_i \sim \rho_i} [-r_i(o_i, a_i)] + \beta^{-1} H(\pi_i),$$

where β is the inverse temperature (related to the entropy coefficient) and $H(\pi_i) = -\mathbb{E}[\log \pi_i]$ is the policy entropy.

Physics analogy — non-equilibrium landscape. In a single-agent setting $\mathcal{F}_i$ is a fixed energy landscape: gradient descent converges to a local minimum. With multiple learning agents, $\mathcal{F}_i$ depends on π_{-i} , which is itself changing. This is the MARL analogue of a non-equilibrium system where the Hamiltonian evolves in time: the ground state is a moving target. The gap between IL convergence and the true cooperative optimum measures how far the system is from equilibrium.

1.4 Independent Learners

Independent Learners (IL) apply a standard single-agent RL algorithm to each agent independently, treating all other agents as part of the (non-stationary) environment. Each agent i maintains a policy $\pi_{\theta_i}(a_i | o_i)$ and updates θ_i using transitions $(o_i^t, a_i^t, r_i^t, o_i^{t+1})$.

Variation 1 — parameter sharing. All agents share the same network weights θ . This is the simplest IL: N agents = N parallel environments feeding the same PPO update. Implemented here via `supersuit.pettingzoo_env_to_vec_env_v1`.

Variation 2 — separate policies. Each agent has its own θ_i . Exposes the non-stationarity signal more clearly; implemented here with a manual REINFORCE loop.

1.5 Centralised Training, Decentralised Execution (CTDE)

CTDE breaks the non-stationarity problem by allowing the *critic* to access global information during training, while the *actor* remains local.

MADDPG (Lowe et al., 2017). For agent i :

Decentralised actor: $a_i = \mu_{\theta_i}(o_i)$

Centralised critic: $Q_{\phi_i}(x, a_1, \dots, a_N), \quad x = (o_1, \dots, o_N)$

The critic is conditioned on the full joint observation-action tuple. At test time the critic is discarded; only μ_{θ_i} runs.

Physics analogy — mean field + local execution.

MARL concept	Physics analogue	Stage
Centralised critic	Mean-field calculation	Training
$Q_{\phi_i}(\mathbf{o}, \mathbf{a})$	Solves coupled spin system	Training
Effective field h_i^{eff}	Output of mean-field	Training
Decentralised actor $\mu_{\theta_i}(o_i)$	Spin reads local h_i^{eff}	Execution

CTDE corresponds to solving the full mean-field equations once at training time, then deploying each spin with its pre-computed effective field. The execution cost is therefore identical to IL.

■

2.1 The `simple_spread` environment

`simple_spread_v3` is a Multi-Particle Environment (MPE) from PettingZoo. N continuous agents (default $N = 3$) in a 2-D bounded world must cover N fixed landmark positions. The reward is:

$$r^t = - \sum_{\ell=1}^N \min_{i=1}^N d(i, \ell)$$

where $d(i, \ell)$ is the Euclidean distance between agent i and landmark ℓ . The negative sign makes reward maximisation equivalent to minimising the total uncovered distance.

Observation space. For $N = 3$ each agent receives an 18-dimensional observation:

- Own velocity (2)
- Own position (2)
- Relative positions of the $N = 3$ landmarks (6)
- Relative positions of the other $N - 1 = 2$ agents (4)
- Communication channel from other agents (4)

Action space. Discrete, 5 actions: idle, move up, move down, move left, move right.

API. We use the **Parallel API**: all agents submit actions simultaneously, mirroring a synchronous physical system.

Why `simple_spread` and not a simpler environment? It is the minimal cooperative benchmark that exhibits all three IL failure modes (lazy agent, collision loop, credit assignment noise) without requiring a GPU or more than a few minutes of training. It also serves as a natural bridge to GABM: the N agents covering N landmarks is a micro-scale version of N synthetic individuals covering N social roles.

Install `pettingzoo[mpe]`, `supersuit`, and `stable-baselines3`. Run once; takes approximately 60 seconds on Colab. Note: `[mpe]` installs the Multi-Particle Environments sub-package.

Imports all required libraries and sets a white-background matplotlib style consistent with the companion slides. Key objects: `simple_spread_v3`, `supersuit` as `ss`, `stable_baselines3.PPO`.

Creates a `parallel_env` instance and resets it. Prints agent names, observation shapes, and action spaces. **What to check:**

- Three agents: `agent_0`, `agent_1`, `agent_2`
- Obs shape: `(18,)` for each
- Action space: `Discrete(5)`
- Observation sample: a vector of floats in roughly `[-1, 1]`

Defines a `run_episode` utility (used throughout the notebook) and runs 1 episode under a random policy. Establishes the **random baseline** reward, stored as `RANDOM_BASELINE`. Typical value: approximately `-1.5` to `-2.0` per step for $N = 3$.

[Cell 2.5] Baseline bar chart

Runs 20 random episodes and plots the per-episode mean reward. The horizontal dashed line is the mean across all episodes. **What to observe:** high variance from episode to episode (stochastic environment initialisation), all values negative (agents always leave some landmark uncovered at

random).

■

3.1 Theory recap (Section 2)

Section 2 of the notebook is a pure markdown cell that derives the effective transition kernel:

$$P_i(s' | s, a_i) = \sum_{a_{-i}} P(s' | s, \mathbf{a}) \pi_{-i}(a_{-i} | s).$$

No code is run. The cell also introduces the free-energy interpretation and connects it to the physics analogy described in §1.3 above.

3.2 Parameter-sharing IL (Section 3)

`make_shared_policy_env` applies four SuperSuit transforms:

1. `pad_observations_v0` — pads all obs to the same shape (required when agents have different obs sizes; harmless here)
2. `pad_action_space_v0` — same for action spaces
3. `pettingzoo_env_to_vec_env_v1` — stacks N agents as N parallel “environments” (one `gymnasium.Env` per agent)
4. `concat_vec_envs_v1` — concatenates into a single SB3-compatible vectorised env with N sub-environments

`VecMonitor` wraps for episode statistics logging.

Hyperparameters:

- `total_timesteps = 200 000` — increase to 500 000+ for better convergence
- `n_steps = 256` — rollout buffer length per sub-environment
- `batch_size = 64, n_epochs = 4`
- `gamma = 0.95` — appropriate for the 25-step episode horizon

Important: all $N = 3$ agents share the *same* network weights. The effective training batch size at each update is $N \times n_steps = 3 \times 256 = 768$ transitions.

Rolls out the trained policy for 30 episodes on a fresh environment. Reports mean $\pm$ standard deviation and the improvement over the random baseline. **Expected outcome:** 20–50% improvement over random, depending on `total_timesteps` and random seed. Parameter sharing works reasonably well because the task is symmetric (all agents face the same observation distribution).

3.3 Separate-policy IL — manual REINFORCE loop (Section 4)

`PolicyNet`: a 2-layer MLP with Tanh activations and a softmax output. Identical architecture to the shared-policy net above.

`discount_returns`: computes normalised discounted returns $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$, normalised to zero mean and unit variance within the episode. Normalisation is standard in REINFORCE to reduce gradient variance.

Each episode:

1. Roll out one episode storing $(\log \pi_i(a_i^t | o_i^t))$ and r_i^t per agent.
2. For each agent i : compute discounted returns, then update $\theta_i \leftarrow \theta_i - \alpha \nabla_{\theta_i} [-\sum_t \log \pi_{\theta_i}(a_i^t) \cdot G_t]$.

Key parameters:

- `N_EPISODES` = 600 — increase to 2 000 for clearer convergence
- `LR` = $3e-4$
- `GAMMA` = 0.95

What to observe:

- Reward should increase from the random baseline but converge slowly
- `reward_var_log` (variance across agents) should remain high — this is the non-stationarity signature

■

4.1 Three diagnostic metrics

The notebook uses three metrics to quantify IL failure in the cooperative setting:

1. **Mean episode reward.** $\bar{r} = \frac{1}{T} \sum_t r^t$, averaged over the episode. Should increase monotonically in a healthy single-agent setting. In IL it typically plateaus early or oscillates.
2. **Between-agent reward variance.** $\sigma_{\text{agents}}^2 = \text{Var}_i [\sum_t r_i^t]$. In a perfectly cooperative team where all agents contribute equally, this should converge to zero. High variance indicates asymmetric learning: one or two agents have “given up” while another dominates.
3. **Per-agent reward distribution.** Histogram of episode-total rewards for each agent over 100 evaluation episodes. A well-coordinated team shows tight, overlapping distributions. A failed team shows bimodal or widely separated distributions.

Physics analogy — order parameter. The between-agent reward variance σ_{agents}^2 plays the role of an *order parameter* for the cooperative phase transition.

- $\sigma_{\text{agents}}^2 \approx 0$: ferromagnetic phase — all agents aligned, cooperating, equally loaded
- $\sigma_{\text{agents}}^2 \gg 0$: disordered phase — broken cooperation, one agent dominates (spontaneous symmetry breaking)

The critical point separating the two phases is controlled by the effective coupling $J \sim \partial r_i / \partial a_j$ (how

strongly one agent's action affects another's reward). In `simple_spread` this coupling is implicit through the shared reward.

Two side-by-side panels:

- **Left:** smoothed (moving average $w = 20$) episode reward for the separate-policy IL, with the random baseline as reference.
- **Right:** smoothed between-agent reward variance over training.

What to discuss with students: the variance plot is the most informative — a system that has learned to cooperate shows decreasing variance; a system stuck in a lazy-agent failure mode shows persistent high variance.

Compares the mean step reward for three policies: random, IL with parameter sharing (SB3 PPO), and IL with separate policies (REINFORCE). Error bars show ± 1 standard deviation over 30 evaluation episodes. **Typical ordering:** $\bar{r}_{\text{shared}} > \bar{r}_{\text{separate}} \approx \bar{r}_{\text{random}}$ for short training runs. The shared policy performs better because parameter sharing is an implicit form of coordination: all agents use the same function approximator, so a good action for agent 0 in state s is also a good action for agent 1 in the same state (by task symmetry).

100 evaluation episodes, per-agent total reward histograms overlaid. **Failure signature:** one agent has a mean reward close to zero (it stays near its starting position because any gradient it receives is washed out by the non-stationary environment), while another agent claims two landmarks. **Success signature** (with enough training or CTDE): the three histograms are tightly clustered around the same mean.

■

5.1 Architecture comparison

Defines the two key neural network components of the CTDE architecture. **Important:**

- `MADDPGActor` is *byte-for-byte identical* to `PolicyNet` defined in Section 4 — same architecture, same input, same output. This drives home the key pedagogical point: CTDE does not change the deployed policy at all.
- `MADDPGCritic` takes the *joint* input $(\mathbf{o}, \mathbf{a}) \in \mathbb{R}^{N \cdot d_o + N \cdot d_a}$. For $N = 3$, $d_o = 18$, $d_a = 5$ this is a 69-dimensional input, versus the 18-dimensional input of the IL critic.
- The parameter count of the critic grows as $\mathcal{O}(N^2)$ (joint input dimension scales linearly in N , but a wider network is also needed).

The cell prints parameter counts to make the trade-off concrete.

A `matplotlib` figure with two panels drawn with `patches` primitives (no external diagrams needed, fully reproducible):

- **Left (IL):** each agent has its own local critic $V_i(o_i)$ connected to the local actor π_i . All connections are solid (same at training and execution).
- **Right (CTDE):** a single centralised critic bar spans all agents. Connections from actors to the critic are dashed (training only, discarded at execution). The actors are unchanged.

Teaching point: the actors in both panels are visually identical. Only the gradient path changes.

■

6.1 From reward to persona

The transition from RL agents to GABM agents involves replacing the gradient-trained policy $\pi_\theta(a | o)$ with an LLM conditioned on a context string: $\pi_{\text{LLM}}(\text{action} | \text{persona} + \text{memory} + \text{goal})$. The MARL interaction structure is preserved:

- **State** → **Environment snapshot** (other agents' positions, resource levels, etc.)
- **Observation** → **Retrieved memory + percepts** (recency $\times$ importance $\times$ relevance retrieval)
- **Action** → **Structured text** parsed to world events
- **Reward** → **Absent**; behaviour emerges from the persona

A pure-Python class (no LLM dependency) that illustrates the data flow of a generative agent:

1. `persona` dict — demographic profile (name, age, income, health, location)
2. `observe()` — appends a timestamped percept dict to the memory stream
3. `_retrieve()` — returns the k most recent entries as a formatted string
4. `act(goal)` — assembles the *prompt* that would be sent to an LLM and prints it (in a real system, calls the API and parses the response)

The cell instantiates two agents (`alice`, `bob`) with different demographic profiles, feeds them observations from a hypothetical COVID-19 model, and prints the prompt for Alice's decision. **Discussion point:** the prompt structure here is a simplified version of the memory-retrieval-reflection-planning pipeline in Park et al. (2023).

A `matplotlib` figure showing the five stages of the full pipeline from single-agent MDP to synthetic population:

Single-agent MDP → Multi-agent IL-PPO → CTDE MADDPG → LLM agent → Synthetic population

Each box is colour-coded (gray → blue → blue → gold → green) to reflect increasing complexity and semantic richness. Below the arrows: labels marking the key challenge introduced at each step. **This diagram is the conceptual map for the rest of the course.**



7.1 Key results from the notebook

1. **Random baseline** establishes $\bar{r} \approx -1.6$ per step for `simple_spread` with $N = 3$.
2. **IL with parameter sharing** (SB3 PPO via SuperSuit) improves by 20–50% over random after 2×10^5 steps. This works because task symmetry makes parameter sharing an implicit coordination mechanism.
3. **IL with separate policies** (REINFORCE) converges slowly and shows persistent between-agent reward variance — the non-stationarity signature. Increasing `N_EPISODES` to 2 000+ may reduce this, but the fundamental problem remains.
4. **CTDE sketch**: the centralised critic sees the joint state; the actor is unchanged. Full training is left as an exercise (implementing a replay buffer and target networks).
5. **GABM bridge**: the LLM agent class shows that the MAS interaction structure is preserved when the policy implementation changes.

7.2 Exercises

1. Increase `N_EPISODES` to 3 000. Does the between-agent variance decrease? What does this tell you about the IL convergence rate?
2. Modify the `PolicyNet` to use a shared *prefix* (common layers) and agent-specific *heads*. How does performance compare to full sharing vs. no sharing?
3. Implement a replay buffer for the separate-policy IL. Does it help or hurt? Justify your answer using the non-stationarity argument.
4. (*Advanced*) Complete the `MADDPGCritic` by adding a target network and a Bellman backup loss. Train for 50 000 steps and compare to IL.
5. The between-agent reward variance is used as a non-stationarity proxy. Can you think of a more direct measure? (Hint: measure the gradient cosine similarity between agents' policy updates.)

7.3 What comes next

The next sessions build on this scaffolding:

- **Generative Agent-Based Models (GABM)** in practice: memory streams, reflection, planning hierarchies.
- **Synthetic populations at scale**: AgentTorch (Chopra et al., 2024) with 10^6+ agents on GPU.
- **Calibration**: fitting synthetic population behaviour to census and mobility data.
- **Phase transitions in GABM**: epidemic thresholds, opinion dynamics tipping points — familiar territory for a physicist.

7.4 References

- Lowe, R. et al. (2017). *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments*. NeurIPS 2017.
- Terry, J. et al. (2021). *PettingZoo: Gym for Multi-Agent Reinforcement Learning*. NeurIPS 2021.
- Park, J.S. et al. (2023). *Generative Agents: Interactive Simulacra of Human Behavior*. UIST 2023.
- Chopra, A. et al. (2024). *AgentTorch: Many-Million Agent Simulations*. arXiv:2310.10997.
- Nash, J. (1950). *Equilibrium Points in N-Person Games*. PNAS 36(1).

■

Symbol	Meaning
N	Number of agents
i, j	Agent indices; $-i$ denotes all agents except i
$\mathcal{S}, \mathcal{A}$	State and action spaces (joint)
o_i, a_i, r_i	Local observation, action, reward for agent i
$\mathbf{o}, \mathbf{a}$	Joint observation and action vectors
π_{θ_i}	Policy of agent i with parameters θ_i
π_{-i}	Joint policy of all agents except i
$Q_{\phi_i}(\mathbf{o}, \mathbf{a})$	Centralised critic for agent i (CTDE)
$V_{\phi_i}(o_i)$	Local value function for agent i (IL)
γ	Discount factor
β^{-1}	Entropy regularisation coefficient (inverse temperature)
$H(\pi_i)$	Entropy of policy π_i
$\mathcal{F}_i$	Local free energy of agent i
h_i^{eff}	Effective field on spin i (mean-field analogy)
J	Spin coupling constant (reward coupling analogy)
σ_{agents}^2	Between-agent reward variance (order parameter)

■

The SuperSuit pipeline used in Section 3 applies four transforms in order. Understanding each step avoids common debugging errors.

Transform	Effect
<code>pad_observations_v0</code>	Pads all agents' observation spaces to the same shape. Required when using <code>concat_vec_envs</code> with heterogeneous agents. Safe to apply even when agents already have the same obs shape.
<code>pad_action_space_v0</code>	Same, for action spaces.
<code>pettingzoo_env_to_vec_env_v1</code>	Converts the PettingZoo Parallel env into a <code>gymnasium.vector.VectorEnv</code> with N sub-environments, one per agent. From this point the env looks like N independent copies of a single-agent Gym env.
<code>concat_vec_envs_v1(env, 1, base_class="stable_baselines3")</code>	Concatenates N sub-envs into a single SB3-compatible vec env. The second argument (1) is the number of additional copies to create beyond the original; set to 1 for a total of N sub-envs.
<code>VecMonitor</code>	Logs episode rewards and lengths; required by some SB3 callbacks.